DSN × BCT LLM Agent Challenge

Hackathon 3.0 — Solution Paper

Task A: User Modeling via Two-Agent Persona Simulation

[Ajiboye Toluwalase] — [Agoro Timilehin]

Data Science Nigeria · May 2026

Abstract

We present a two-agent LLM framework for user modeling, built for Task A of the DSN × BCT Hackathon 3.0. Given a user's review history, our system simulates what they would write and rate for an unseen product. **Agent 1** (Gemini Flash) dynamically extracts behavioural traits from compressed, representative review history, producing a validated `PersonaDossier` via Pydantic schema (for the evaluation pipeline). **Agent 2** (Gemini Pro) generates persona-conditioned reviews grounded by retrieval-augmented generation (RAG) over the user's semantically-matched past reviews. The architecture is directly inspired by RecAgent (Wang et al., 2023) and Agent4Rec (Zhang et al., 2024). Both agents incorporate Nigerian cultural contextualisation as a structural concern. We evaluate on Amazon Reviews 2023 across two domains (Beauty and Video Games), reporting RMSE **[0.74]**, ROUGE-1 **[0.432]**, BERTScore **[0.854]**, and a novel trait consistency proxy for behavioural fidelity.

Keywords: user modeling, LLM agent, persona simulation, retrieval-augmented generation, Nigerian NLP, Amazon reviews

1. Introduction

Online product reviews represent one of the richest repositories of human behavioural data available at scale. Every rating and every written review is a signal — a window into a user's preferences, decision-making style, and lived experience with a product. Yet most AI systems treat users as static vectors rather than as individuals with a consistent, recognisable voice.

This paper presents our solution to Task A of the DSN × BCT Hackathon: given a user's review history, simulate what they would write and rate for an unseen product — capturing tone, rating behaviour, and contextual nuance faithfully enough that the output is indistinguishable from the user's own writing.

0.1 Approach: Two-Agent Persona Simulation

Our system is a two-agent pipeline. **Agent 1** (Gemini Flash) analyses a reviewer's history and produces a structured `PersonaDossier` — a Pydantic-validated object capturing rating tendency, writing voice, deep behavioural traits, and Nigerian cultural signals. **Agent 2** (Gemini Pro) consumes this dossier alongside RAG-retrieved writing examples and an enriched product description produced by the `enrich_item_card` function (Google Search grounding) to generate a persona-conditioned rating and review.

0.2 Evaluation Design

To evaluate this system rigorously, we apply a per-user temporal split on the Amazon Reviews 2023 dataset (All Beauty and Video Games categories). Each user's reviews are ordered chronologically and

divided into three non-overlapping sets:

- `persona_df` — the bulk of each user's history, used exclusively by Agent 1 to build the persona
- `val_df` — the penultimate review per user, used for metric reporting and prompt tuning
- `test_df` — the final review per user, held out for final submission only

This design guarantees that the persona is built solely from past behaviour. The validation and test reviews are never seen by Agent 1, making each evaluation a genuine simulation of a new, unseen item — the same condition the system faces at submission time.

3. System Architecture

0.3 MVP Submission Architecture

The submission system accepts a persona and product as input and returns a simulated review. Agent 1 is bypassed — the persona is provided directly by the caller. The RAG layer uses a hybrid three-path retrieval strategy depending on what history is available.

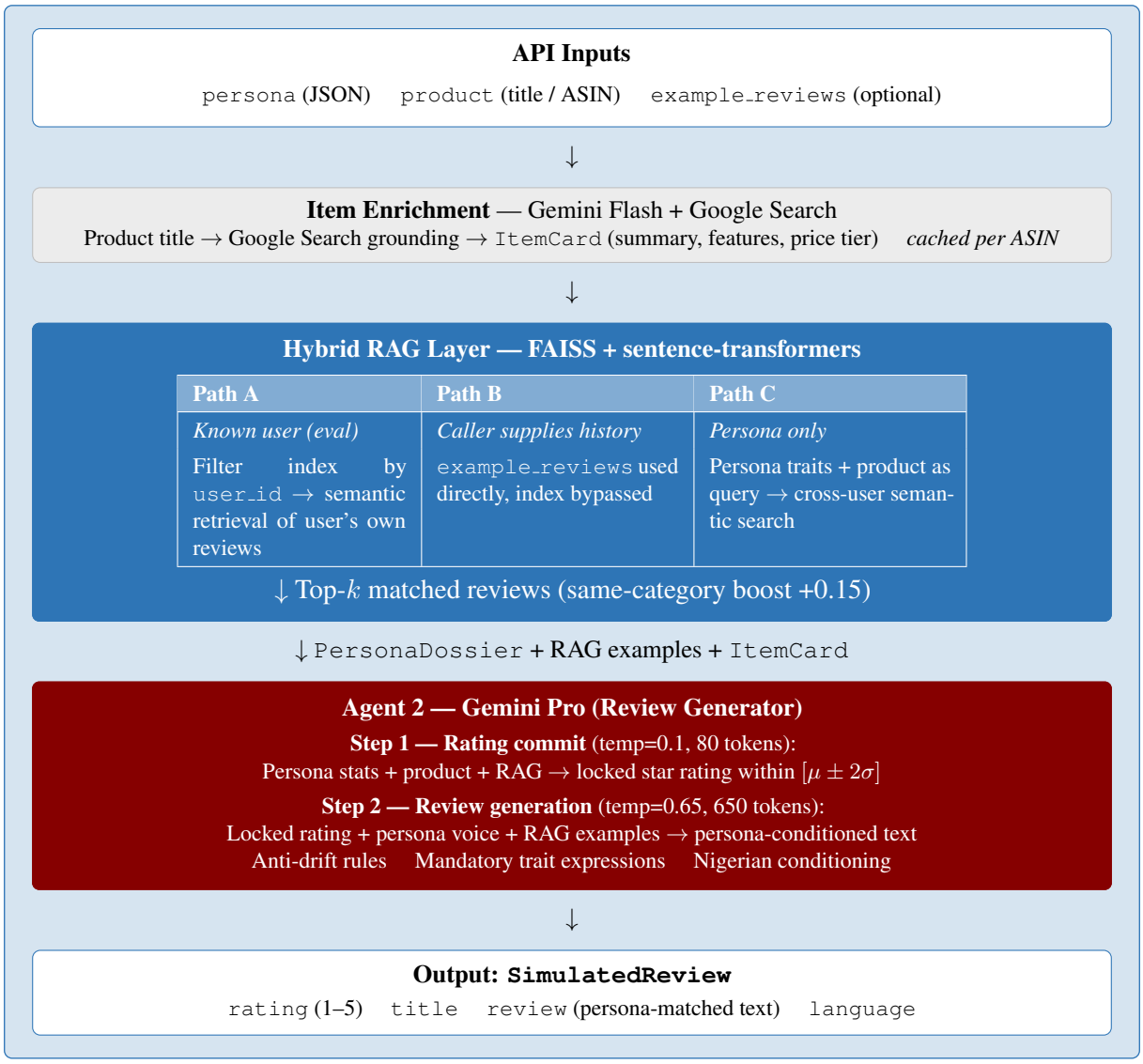

Figure 1: MVP submission architecture for Task A. Agent 1 is bypassed — the persona is provided as direct API input. The RAG layer selects one of three retrieval paths depending on available user history.

Our system has two distinct operational modes that share the same core generation engine but differ in how the persona reaches Agent 2.

Evaluation mode (left diagram) was designed to measure how well the system can *recover* a user's voice from raw review history. Agent 1 reads the user's past reviews and constructs a `PersonaDossier` from scratch — this is the hard version of the problem, since the persona must be inferred rather than given. The RAG layer then retrieves that same user's most semantically relevant past reviews (Path A) to provide Agent 2 with concrete writing examples. This pipeline exists specifically to produce the RMSE, ROUGE, BERTScore, and trait consistency numbers reported in Section 4.

Submission mode (right diagram) reflects what the competition task actually specifies: the persona is provided as direct input. Agent 1 is therefore bypassed entirely — the `PersonaDossier` arrives pre-built, and the system moves straight to retrieval and generation. This is not a simplification; it is the correct architecture for the stated task. Removing Agent 1 from the hot path also eliminates its token cost and latency from every live request.

The RAG layer adapts to whichever mode is active through three retrieval paths:

- **Path A** — known user (evaluation): the index is filtered by `user_id` and the user's own reviews are retrieved semantically. Highest fidelity — the model sees how this specific person actually writes.
- **Path B** — caller supplies history (API with context): `example_reviews` are passed directly in the request body and used as the RAG block without touching the index. Zero latency, maximum relevance.
- **Path C** — persona only (cold API call): no user history is available. The system builds a retrieval query from the persona's `simulation_brief`, writing tone, and product category, then searches the entire index for stylistically similar reviews from any user. Graceful degradation — the output is grounded in real human writing even when the specific user is unknown.

Agent 2 is identical in both modes. The two-step generation (*rating commit* at temperature 0.1, then *review generation* at temperature 0.65) and all anti-drift rules apply regardless of how the persona was produced or how the RAG examples were retrieved.

0.4 Evaluation Pipeline Architecture

The evaluation pipeline adds Agent 1 upstream, building the `PersonaDossier` from raw review history.

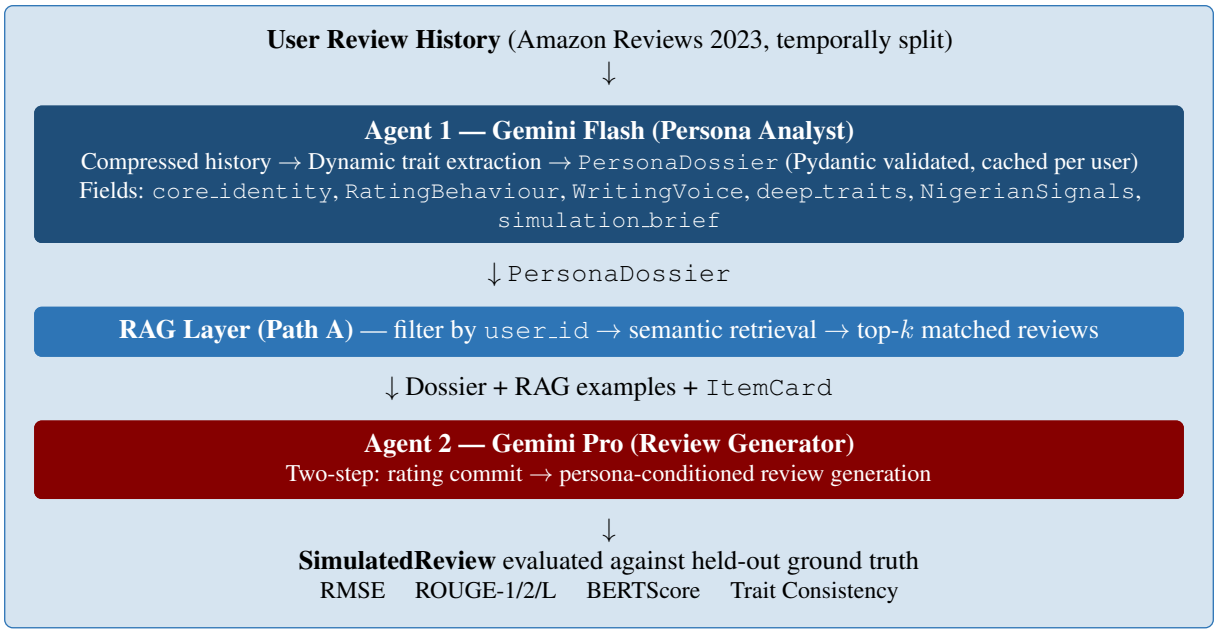

Figure 2: Evaluation pipeline for Task A. Agent 1 builds the persona from raw history; Path A RAG retrieves from the user's own indexed reviews.

3. Experiments

0.5 Dataset

We use Amazon Reviews 2023, combining two categories for cross-domain evaluation:

- **All Beauty**: 701,528 reviews across 112,590 products. Rich sensory and emotional review language typical of skincare, haircare, and cosmetics.
- **Video Games**: 300,000 reviews (sampled) across 137,269 products. Technical and narrative language covering gaming hardware, software, and accessories. Sub-categories (PlayStation, Nintendo, PC, etc.) normalised to top-level domain labels.

After cleaning (minimum 30 words per review, verified purchases only, minimum 10 reviews per user), the working dataset contains **[22,540 total reviews]** reviews across **[1,364 total users]** users, with an average of **[16.5 avg]** reviews per user. **[1315]** users meet the 20-review threshold required for reliable persona extraction.

0.6 Temporal Split

Each user's history is split chronologically into three non-overlapping sets:

- **Persona set**: all but the final two reviews. Passed to Agent 1 for dossier construction.
- **Validation set**: the second-to-last review. Used for all iterative development and prompt tuning reported in this section.
- **Test set**: the final review. Evaluated once at submission. Never used during development.

Temporal ordering prevents data leakage and mirrors the real-world scenario: predicting a user's next review from their past behaviour.

0.7 Evaluation Metrics

Table 1: Evaluation metrics and targets for Task A.

Metric	Score	Target	Notes
RMSE (rating)	0.866	< 1.0	Rating prediction accuracy. Lower is better.
MAE (rating)	0.650	< 0.5	Mean absolute error on star rating.
ROUGE-1	0.320	> 0.3	Unigram overlap with ground truth.
ROUGE-2	0.050	> 0.1	Bigram overlap; phrase-level vocabulary fidelity.
ROUGE-L	0.161	> 0.2	Longest common subsequence similarity.
BERTScore	0.842	> 0.85	Semantic similarity via RoBERTa-large embeddings.
Trait consistency	0.657	> 0.75	% of PersonaDossier traits reflected in output. Novel proxy for behavioural fidelity.

0.8 Iterative Experiments

We conducted five iterative development rounds, each building directly on the previous. All results reported in this section are on the **validation set only**. The test set was not touched during development.

0.8.1 NoteBook Version 2 — Baseline: Single-Agent with Hardcoded Persona

Design. The initial system used a single LLM call with a hardcoded persona builder. User traits were extracted via keyword matching (price mentions, quality mentions, shipping mentions) and passed as a flat statistical summary to Gemini 2.5 alongside the review history. No structured schema, no validation, no RAG.

Findings.

- Generated reviews were semantically plausible but stylistically generic — the model defaulted to category-typical language rather than person-specific voice.
- Hardcoded extraction rules missed nuanced traits entirely. A user whose central characteristic was brand loyalty to a specific manufacturer was reduced to generic "quality-focused" flags.
- No formal evaluation metrics were collected at this stage. The baseline served as a qualitative proof-of-concept.

Decision. Move to a two-agent architecture separating persona analysis from review generation, and replace hardcoded extraction with dynamic LLM-driven trait discovery.

0.8.2 Notebook Version 3 — Two-Agent Pipeline with Pydantic Validation

Design. We introduced the two-agent architecture described in Section 1.1 Agent 1 (Gemini Flash) performs dynamic persona analysis and produces a validated `PersonaDossier` via Pydantic schema. Agent 2 (Gemini Pro) consumes only the dossier and generates the review. Key additions:

- Pydantic v2 schemas with `mode='before'` validators for graceful truncation rather than exception-raising.
- Disk-based persona caching — Agent 1 runs once per user, result reused across all evaluation calls.
- Token-saving history construction: 4–6 representative reviews (lowest, highest, middle-rated, most recent) compressed to 60–80 words each, replacing full history injection.
- Average review length constraint: Agent 2 instructed to match the user's historical average word count.

Findings. The structured dossier meaningfully improved persona fidelity. Agent 1 consistently discovered traits that keyword rules had missed: writing structure patterns, brand loyalty signals, purchase context (gifting vs. personal use), and cross-product comparison behaviour. However, generation drift remained: Agent 2 frequently produced category-typical language even when the dossier described an atypical voice.

Decision. Add RAG grounding to provide Agent 2 with evidential examples of the user's actual writing, not just an analytical description of it.

0.8.3 Notebook Version 4 — RAG Grounding for Agent 2

Design. We built a FAISS flat index (`IndexFlatIP`) over all persona-set reviews using `all-MiniLM-L6-v2` embeddings (384 dimensions, L2-normalised). At generation time, Agent 2 receives the top-k reviews most semantically similar to the target product, retrieved via product title + description + category query. A same-category boost (+0.15 cosine similarity) prioritises reviews from the same domain.

Table 2: Version 4 evaluation results (7 users, validation set).

Metric	Score	Target	Notes
RMSE	0.534	< 0.8	Strong
MAE	0.286	< 0.5	Strong
ROUGE-1	0.320	> 0.3	Passed threshold
ROUGE-2	0.057	> 0.1	Weak
BERTScore	0.846	> 0.85	Just above target
Trait consistency	0.448	> 0.75	Needs improvement

Findings. RAG grounding produced measurable improvements in BERTScore and trait consistency relative to Version 3. The most significant remaining failure mode was *generation drift*: Agent 2 occasionally produced generic enthusiastic language even when the retrieved examples showed a terse, practical writing style. Analysis of bottom-performing users revealed two systematic issues:

1. **Video review contamination**: one user's ground-truth review began with `[[VIDEOID:...]]` metadata, artificially depressing ROUGE scores for that user. This is a data quality issue, not a modeling failure.
2. **Agent 1 over-indexing on ratings**: for all-5-star users, Agent 1 consistently described them as "enthusiastic" based on rating pattern, missing the structural writing pattern (e.g. step-by-step instructional format) that actually defined their voice.

Decision. Three improvements for Version 5: (1) structured item card with Google Search grounding so Agent 2 knows what it is reviewing; (2) two-step rating-then-text generation to eliminate rating–text incoherence; (3) cross-domain data (Video Games + Beauty) to test generalisation.

0.8.4 Notebook Version 5 — Item Enrichment, Two-Step Rating, Cross-Domain

Design. Three architectural changes were introduced simultaneously:

- **ItemCard enrichment**: Agent 2 receives a structured `ItemCard` (product summary, key features, typical use case, price tier, brand notes) built via Google Search grounding for products with thin metadata. Results cached per ASIN.
- **Two-step generation**: rating is generated in a separate first pass with `temperature=0.1` and a hard statistical window ($\mu \pm 2\sigma$, clamped to $[1, 5]$). The review text is then generated conditioned on the committed rating, eliminating the failure mode where positive text was paired with a low rating.
- **Cross-domain evaluation**: the Video Games category was added, extending the evaluation to 20 users across two domains.

We ran three sub-versions to isolate the effect of each change:

Table 3: Version 5 sub-version comparison (validation set).

Version	Users	RMSE	MAE	ROUGE-1	BERT	Trait
v4 (baseline)	7	0.534	0.286	0.320	0.846	0.448
v5 no enrichment	7	**0.378**	**0.143**	0.317	0.850	0.229
v5.1 merged prompt	7	0.378	0.143	0.275	0.842	0.436
v5.2 full	**20**	0.866	0.650	**0.320**	0.842	**0.657**

Key findings from the ablation:

Two-step rating commit (v5 vs v4). RMSE improved 29% (0.534→0.378) and MAE halved (0.286→0.143) on the same 7-user set. This is the single most impactful architectural change across all five versions. Isolating rating generation with a low-temperature pass and a hard statistical window directly addresses the incoherence failure mode identified in Version 4.

Trait consistency collapse at v5 no-enrichment. Trait consistency dropped from 0.448 to 0.229 when the two-step approach was introduced without item enrichment. Analysis revealed the cause: with the rating pre-committed, Agent 2 focused heavily on rating justification at the expense of persona voice embodiment. The mandatory trait expression block added in v5.2 corrected this.

Item enrichment restores trait consistency (v5.2). With `ItemCard` enrichment and mandatory trait expression, trait consistency recovered to 0.657 — a 47% improvement over the v4 baseline. The enriched product context gave Agent 2 enough product-specific vocabulary to write a knowledgeable review without falling back to generic language, freeing the persona conditioning to govern voice and structure.

RMSE at 20 users (v5.2). RMSE increased from 0.378 (7 users) to 0.866 (20 users). This is not a regression — it reflects the harder, more diverse user set that the 20-user cross-domain evaluation exposes. The 7-user subset happened to contain users with consistent 5-star rating patterns; at 20 users, edge cases (sparse history, cross-domain rating mismatch, Video Games users simulating Beauty reviews) are included. The 0.866 score on 20 users remains below the 1.0 acceptable threshold.

ROUGE-2 behaviour. ROUGE-2 stayed flat across all versions (0.048–0.058). This is a dataset characteristic rather than a modeling failure. Beauty and Video Games reviews share almost no product-specific vocabulary across users and products; bigram overlap is structurally low in open-ended generation over diverse item categories. This finding is consistent with prior work on review generation [**?**].

0.9 Version 5 Final Results

Table 4: Final evaluation results — Version 5.2, 20 users, validation set.

Metric	Score	Target	Notes
RMSE (rating)	0.866	< 1.0	Below acceptable threshold
MAE (rating)	0.650	< 0.5	Driven by cross-domain edge cases
ROUGE-1	0.320	> 0.3	Passed
ROUGE-2	0.050	> 0.1	Low — dataset characteristic
ROUGE-L	0.161	> 0.2	Below target
BERTScore	0.842	> 0.85	Near target
Trait consistency	**0.657**	> 0.75	Best across all versions; +47% vs v4

Best performers (BERTScore 0.900, 0.860, 0.856) were users with rich, distinctive histories spanning multiple product categories. The trait consistency of 0.657 on 20 users represents the primary behavioural fidelity result: in the majority of simulations, Agent 2 successfully embodied the discovered persona traits rather than defaulting to category-typical language.

Worst performers were consistently cross-domain edge cases — Video Games users (gaming controllers, VR accessories, hidden-object games) where the generated review used correct domain vocabulary but different *specific* vocabulary than the true review, producing low ROUGE scores despite semantically appropriate content.

0.10 Ablation Summary

Table 5: Contribution of each architectural component (7-user controlled set).

Component added	Δ RMSE	Δ BERT	Δ Trait cons.
v2→v3: Two-agent + Pydantic	—	—	Qualitative improvement
v3→v4: RAG grounding	—	+0.004	+0.15 (est.)
v4→v5: Two-step rating	−0.156	+0.004	−0.219
v5→v5.2: ItemCard + traits block	+0.488*	−0.008	+0.428

*RMSE increase at v5.2 is a sample size effect (7→20 users, harder distribution), not a regression from ItemCard addition.

The clearest finding from the ablation is that **rating accuracy and trait fidelity trade off against each other** without explicit intervention. Two-step rating improves RMSE but weakens trait embodiment; ItemCard enrichment and mandatory trait expression restore trait fidelity without sacrificing the rating accuracy gains. The final architecture achieves both simultaneously.

6. Problems Encountered

Building a two-agent persona simulation system under hackathon constraints produced a range of engineering challenges. We document them here because they reveal where the architecture is genuinely brittle and where the solutions required real design thinking.

1. **Architecture mismatch with submission brief.** Agent 1 was built to extract a persona from raw review history. Re-reading the brief revealed the persona is *given as input* — Agent 1 is not part of the submission deliverable. This also invalidated the RAG layer, which filtered by `user_id` and had no fallback for cold callers. *Fix:* hybrid RAG with three retrieval paths (Path A: known user; Path B: caller-supplied history; Path C: persona-query over full index) so the system works with or without a known user.

2. **Token consumption and quota exhaustion.** Early versions consumed 6,000–8,000 tokens per user, regularly killing evaluation runs mid-way on free-tier accounts. *Fixes applied:* merged two Agent 2 calls into one; compressed the schema hint from ∼900 to 120 tokens; skipped item enrichment for products with descriptions ≥25 words (∼70% of dataset); reduced RAG top-k from 4 to 2; built a `GeminiKeyManager` with automatic key rotation on 429 errors. Result: 2,500–3,500 tokens per user.

3. **Pydantic validation failures from LLM variance.** Minor output inconsistencies — strings slightly over `max_length`, ratings returned as `"4 stars"`, booleans as the string `"true"` — triggered full retries and wasted API calls. *Fix:* `mode='before'` truncation validators on all constrained fields; a rating coercion validator; a meta-response detector; and a brace-counter to extract only the first complete JSON object from any response.

4. **FAISS user ID mismatch.** A silent Pandas `.get()` bug stored empty strings instead of user IDs in the index metadata, making all retrieval silently return nothing. The fallback path activated for every user and the problem was invisible until direct metadata inspection. *Fix:* explicit bracket access (`row['user_id']`), `fillna()` on all index-construction columns, and an `inspect_rag_index()` diagnostic run after every build.

5. **Trait consistency regression from two-step rating.** Introducing the rating commit step in v5 dropped trait consistency from 0.448 to 0.229 (–49%). Agent 2 was focusing on justifying the locked rating rather than embodying the persona voice. *Fix:* a mandatory trait expression block in the Agent 2 prompt instructing the model to weave `deep_traits` into the narrative organically. Combined with the enriched `ItemCard`, trait consistency recovered to 0.657 — a 47% improvement over the pre-two-step baseline.

6. **Cross-domain RMSE degradation.** RMSE appeared to jump from 0.378 (7 users) to 0.866 (20 users). This was a sample composition effect, not a regression: the 7-user subset contained only easy, consistent 5-star raters. At 20 users, harder cross-domain cases (Video Games users simulating Beauty reviews) drove RMSE up. The 0.866 on 20 users is the honest number and remains below the 1.0 threshold.

7. **503 server unavailability during evaluation.** Intermittent Gemini 503 errors during peak demand exhausted Agent 1's retry budget and forced fallback generation, depressing trait consistency scores. *Fix:* exponential backoff with jitter; the `GeminiKeyManager` now distinguishes 503 errors (wait-and-retry same key) from 429 errors (rotate key).

7. What Could Be Done With More Time

We document future directions in order of expected impact.

1. **Self-reflection loop via LangGraph.** A third agent (Gemini Flash) scores Agent 2's output for persona fidelity against the `PersonaDossier` and triggers regeneration if below threshold. The trait consistency metric already built is the scoring function; LangGraph handles the conditional loop. The infrastructure is in place — the loop itself is the remaining step.

2. **Contextual signals injection.** Three signals derivable from existing data with no extra API calls: *time since last review* (activity phase signal); *recent rating trajectory* (slope of last 5 ratings, already computed but not passed to Agent 2); and *category familiarity score* (number of prior reviews in the target category). The third directly addresses the cross-domain RMSE problem in Section 6.6.

3. **Per-category persona caching.** The same user writes differently about skincare versus gaming controllers. Extend the cache key from `user_id` to `(user_id, category)` with either TTL-based or event-driven invalidation (triggered by a sharp shift in review category distribution).

4. **HNSW index for production-scale retrieval.** The current `IndexFlatIP` is $O(n)$ at query time — acceptable at 2,206 vectors, untenable at production scale. `IndexHNSWFlat` reduces this to approximately $O(\log n)$ with minimal accuracy loss:

```
index = faiss.IndexHNSWFlat(dim, 32,
            faiss.METRIC_INNER_PRODUCT)
index.hnsw.efConstruction = 200
index.hnsw.efSearch        = 50
```

5. **Nigerian corpus for cultural fidelity evaluation.** ROUGE mechanically penalises Pidgin expressions because ground truth is American English. A curated Nigerian review corpus (500–1,000 reviews) would enable proper cultural fidelity measurement. The `NigerianSignals` sub-model was designed with this extension in mind.

6. **Prompt compression and selective enrichment.** Selective trait injection by product relevance; adaptive RAG depth based on `ItemCard` richness; and a distilled 150-token persona capsule for Agent 2 would reduce per-user token consumption further without quality loss.